QUIC PROTOCOL ANALYSIS REPORT

by Rajan Hasija

OVERVIEW

QUIC is a

- secure general-purpose transport protocol.
- connection-oriented protocol – creates stateful interaction between server and client

Quick Handshake combines negotiation of **Cryptographic** + **Transport** parameters.

Includes option for clients to send data immediately **(0-RTT)** (prior communication/configuration required)

Application protocols exchange information over a QUIC Connection via **streams** (**Ordered sequence of bytes**).

QUIC depends on congestion control to avoid network congestion.

QUIC connections are **not strictly bounded** to a single network path. Connection migration uses **Connection IDs** to allow connections to transfer to a new network path (**Only clients** can migrate in Version 1)

Current versions:

- Version 1 (RFC 9000): 0x00000001 (**Version-Independent Properties**)
- Version 2 (RFC 9369): 0x6b3343cf

STREAMS

Application protocols exchange information over a QUIC Connection via **streams** (**Ordered sequence of bytes**). They last the **entire** duration of the connection.

Two types of streams:

i) Bidirectional streams: Allow both endpoints to send data
ii) Unidirectional streams: Allow a single endpoint to send data

Identified by a numeric value, called **Stream ID** (62-bit integer), unique for all streams on a connection. They are encoded as Variable-Length Integers. (Must NOT be reused within a connection).

- **LSB** of Stream ID is the **initiator** (0 – Client, 1 – Server)
- **Second** LSB of Stream ID identifies **Unidirectional** (1) / **Bidirectional** (0) Stream.

CONNECTION

Connection starts with **Handshake Phase**:

- The two endpoints established a shared secret key using the cryptographic handshake protocol.
- This handshake confirms that both endpoints are willing to communicate and establishes parameters for connection.

CLIENT -> **Initial [0]:** CRYPTO (Client HELLO)-> **SERVER**
SERVER -> **Initial [0]:** CRYPTO (Server HELLO) **& Handshake [0]:** CRYPTO -> **CLIENT**
CLIENT -> **Handshake [1]:** CRYPTO (Finished) -> **SERVER**
SERVER -> **1-RTT** Data (Encrypted) -> **CLIENT**

An application protocol can use the connection during the handshake phase with some limitations:

- RTT allows application data to be sent by a client before receiving a response from a server. However, 0-RTT does **NOT** provide any protection against replay attacks.
- A server can also send application data to a client before it receives the final cryptographic handshake messages that allow it to confirm the liveness and identity of the client.

These capabilities allow the application protocol to offer the option of trading some security guarantees for reduced latency.

Connections IDs allow connections to migrate to a new network path, both by choice of an endpoint, and when forced to change in the middlebox.
Primary function: To ensure that the changes happening at the lower protocol layers (UDP, IP) does not cause the QUIC packets to be delivered to the wrong endpoint.

PACKET FORMATS

All numeric values are encoded in network byte order, and all field sizes are in bits (in hexadecimal notation).

Packet Numbers: **$0 - 2^{62} - 1$ (62-bit)**

Encoding: In long or short packet headers, they are encoded in 1 to 4 bytes. Only least significant bits of the packet number are included to reduce the size. This is then encrypted.

Decoding: At receiver end, first the protection of the packet number is removed. The full packet number is reconstructed based on:

I. No. of significant bits present
II. Largest packet number received

i) **Long Header Packets: Source Connection ID + Destination Connection ID (for new connections) –** Allows special packets (Version Negotiation packet)

```
Long Header Packet {
  Header Form (1) = 1,
  Fixed Bit (1) = 1,
  Long Packet Type (2),
  Type-Specific Bits (4),
  Version (32),
  Destination Connection ID Length (8),
  Destination Connection ID (0..160),
  Source Connection ID Length (8),
  Source Connection ID (0..160),
  Type-Specific Payload (..),
}

               Figure 13: Long Header Packet Format
```

ii) **Short Header Packets: Only Destination Connection ID**

- **Long Headers** are used for packets sent prior to establishment of 1-RTT keys. Once 1-RTT keys are available, the sender switches to **short header**.
- Once header protection is removed, the packet number is decoded by finding the packet number value closest to the next expected packet (Highest received packet no. + 1).

VERSION NEGOTIATION

Allows server to indicate that it does not support the version the client used, by sending a Version Negotiation Packet to any packet that might initiate a new connection.

Determined by the **first packet** sent by a client.

For clients that support multiple QUIC versions, they must ensure that the first UDP datagram sent is sized: **LARGEST**(**MINIMUM** (Datagram sizes supported for each version))

The Version Negotiation packets includes a list of versions accepted by the server.

```
Version Negotiation Packet {
  Header Form (1) = 1,
  Unused (7),
  Version (32) = 0,
  Destination Connection ID Length (8),
  Destination Connection ID (0..2040),
  Source Connection ID Length (8),
  Source Connection ID (0..2040),
  Supported Version (32) ...,
}

              Figure 14: Version Negotiation Packet
```

QUIC vs Generic UDP

1. Packet Header Structure Differences

Generic UDP (42 Bytes): Simple

- Ethernet Header (14 bytes)
- IP Header (20 bytes for IPv4)
- UDP Header (8 bytes)
- Payload (Variable Length)

QUIC:

- Ethernet Header(14 bytes)
- IP Header (20 bytes for IPv4)
- UDP Header (8 bytes)
- QUIC Header (10-50+ bytes)
- QUIC Payload (Encrypted)

Key Difference: QUIC has a substantial header with identifiable structure

2. QUIC Header Patterns

First Byte (QUIC):

- Header Form (H): 0 = short, 1 = long
- Fixed Bit (F): 1 for QUIC
- Type: Packet Type
- Reserved (R)
- Packet Number Length (P)

First Byte (Generic UDP): Any value, no consistent pattern

Version Field (Long Header Only):

- 0x00000001 – QUIC v1
- 0x6b3343cf – QUIC v2
- 0x00000000 – Version Negotiation

Version Field (Generic UDP): ABSENT

Connection ID: Variable Length (0-20 bytes), absent in Generic UDP

3. Packet Size Distributions

QUIC Traffic Patterns

A. QUIC Handshake:

Packet 1 (Initial – Client Hello): 1200-1280 bytes
Packet 2 (Initial – ServerHello): 1200-1280 bytes
Packet 3-4 (Handshake): 1000-1200 bytes
Packet 5+ (1-RTT Data): Highly Variable Size (200-1400 bytes)

Observable pattern:

1. Initial burst of large packets (~1200 bytes)
2. Then variable-sized data packets

B. QUIC Web Browsing:

Typical size distribution:

- Small (< 300 bytes): (25%)
- Medium (300-800 bytes): (35%)
- Large (800-1400 bytes): (40%)

Pattern: Mixed sizes, bursty
Reason: HTTP requests are small, but responses and downloads are large

C. QUIC Streaming

Typical size distribution:

- Small (< 300 bytes): (5%)
- Medium (300-800 bytes): (10%)
- Large (800-1400 bytes): (85%)

Pattern: Dominated by large packets, steady stream
Reason: In video chunks, almost all packets are full sized, steady transmission rate

Generic UDP Traffic Patterns

A. DNS (Port 53)

- Query packets: 40-100 bytes
- Response packets: 50-512 bytes

Pattern: Small, paired query-response

B. **NTP (Port 123)**

All packets: EXACTLY 90 bytes (IP+UDP+NTP)

Pattern: Fixed size, periodic

C. **VoIP/RTP (Various Ports)**

All packets: ~200-220 bytes (very consistent)

Pattern: Constant rate, fixed size

D. **Online Gaming (Various Ports)**

Tiny packets: 40-150 bytes

Pattern: Very small, high frequency

4. Timing Patterns (Inter-Arrival Times)

QUIC Traffic Timing

A. **QUIC Web:**

Time between packets (milliseconds):

Handshake phase:
- Packet 1 → Packet 2: ~50ms (RTT)
- Packet 2 → Packet 3: ~1ms (burst)
- Packet 3 → Packet 4: ~1ms (burst)

Data phase:
- Highly variable: 0.1ms - 100ms

Bursty pattern: groups of packets close together

B. **QUIC Streaming:**

Time between packets:

Steady stream: ~5-20ms between packets
Occasional bursts: 0.1-1ms (during buffering)
Very consistent delivery rate

Generic UDP Timing

A. DNS:

- Query at t=0
- Response at t=50ms
- [Silence for minutes]
- Next query at t=300000ms

Pattern: Query-response pairs separated by long silence

B. NTP:

- Request at t=0
- Response at t=50ms
- [Silence for ~5 minutes]
- Next request at t=300000ms

Pattern: Periodic, widely spaced

C. VoIP:

- Packets every 20ms exactly

Pattern: Clock-like precision, constant rate

5. Port Usage Patterns

QUIC

Primary ports:

- UDP 443 (HTTPS) 95%
- UDP 80 (HTTP) 3%
- UDP 8080, 4433, etc. 2%

Pattern: Strongly concentrated on port 443

Generic UDP

Common ports:

- UDP 53 (DNS): Frequent
- UDP 123 (NTP): Periodic
- UDP 67/68 (DHCP): Occasional
- UDP 161 (SNMP): Monitoring
- UDP 5060 (SIP): VoIP

- UDP 3478 (STUN): WebRTC
- Random high ports: Gaming, P2P

Pattern: Widely distributed across port range

Key difference:

- QUIC: Almost always port 443
- Generic UDP: Spread across many ports

Environment Setup Guide

Here are the steps to set up the **IETF QUIC** protocol stack, traffic generation tools, and packet capture permissions.

1. Project Directory Structure

To maintain consistency across scripts and ML models, all team members should use the following layout:

Plaintext
```
QUIC-Project/
├── libs/        # Compiled QUIC implementations (ngtcp2, quiche)
├── data/        # Raw PCAPs and SSL Keylogs (Add to .gitignore)
├── scripts/     # Python automation and ML code
└── docs/        # Project documentation
```

2. System Dependencies

Install the necessary compilers and networking libraries for your specific OS:

- **macOS:** brew install cmake ninja rust libev openssl@3 pkg-config
- **Linux/WSL2:** sudo apt update && sudo apt install -y cmake ninja-build rustc libev-dev libssl-dev pkg-config

3. Building the Protocol Stack

Using **ngtcp2** as our primary reference for IETF QUIC traffic.

A. Build nghttp3 (The HTTP/3 Layer)

Bash
```bash
cd QUIC-Project/libs
git clone --recursive https://github.com/ngtcp2/nghttp3
cd nghttp3
autoreconf -i
./configure --prefix=$PWD/build --enable-lib-only
make -j$(nproc 2>/dev/null || sysctl -n hw.ncpu) install
```

B. Build ngtcp2 (The QUIC Layer)

Bash
```bash
cd ../
git clone --recursive https://github.com/ngtcp2/ngtcp2
cd ngtcp2
autoreconf -i

# For Linux/WSL:
PKG_CONFIG_PATH=$PWD/../nghttp3/build/lib/pkgconfig ./configure --with-openssl
```

```
# For macOS (Apple Silicon):
./configure \
  LIBEV_CFLAGS="-I/opt/homebrew/include" \
  LIBEV_LIBS="-L/opt/homebrew/lib -lev" \
  PKG_CONFIG_PATH="$PWD/../nghttp3/build/lib/pkgconfig" \
  --with-openssl

make -j$(nproc 2>/dev/null || sysctl -n hw.ncpu)
```

4. Verification & Key Logging

The goal of this project is to identify traffic. To verify your model's accuracy, you must be able to decrypt the traffic using an **SSLKEYLOGFILE**.

The Smoke Test

Run the following commands to confirm your client can connect to the internet and log its secrets:

1. **Create the data directory:**
   `mkdir -p ~/QUIC-Project/data`
2. **Run the client:**
   Bash
   ```
   export SSLKEYLOGFILE=~/QUIC-Project/data/keys.log
   ./examples/osslclient cloudflare-quic.com 443
   ```
3. **Confirm the Log:**
   `cat ~/QUIC-Project/data/keys.log`
   (You should see lines starting with `CLIENT_HANDSHAKE_TRAFFIC_SECRET`)

5. Packet Capture Permissions

To allow Python scripts to capture traffic without root access:

- **macOS:** Open Wireshark → Menu → **Install ChmodBPF**.
- **Linux:** Run `sudo usermod -aG wireshark $USER` then log out and back in.

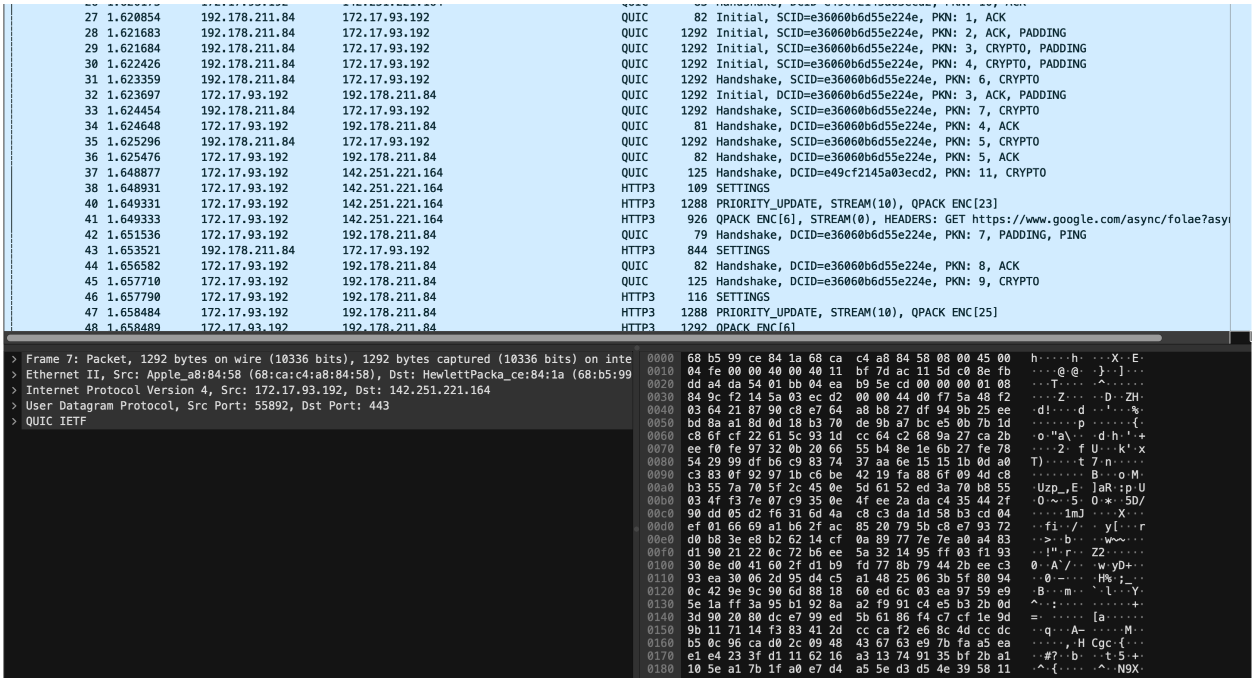
Troubleshooting Tips

- **Binary Names:** On macOS/Linux, the client is usually located at `libs/ngtcp2/examples/osslclient`.
- **MTU Issues:** If you see `sendmsg: Message too long`, it is normal. QUIC is performing Path MTU Discovery (PMTUD).

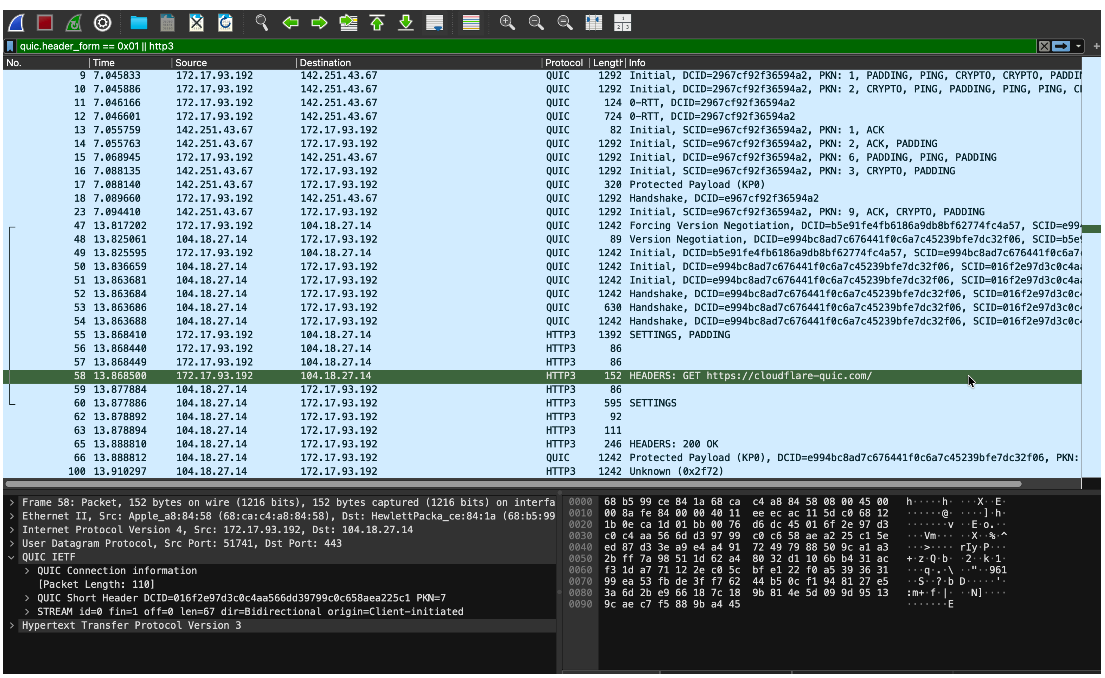
TRAFFIC CAPTURE

QUIC DATA

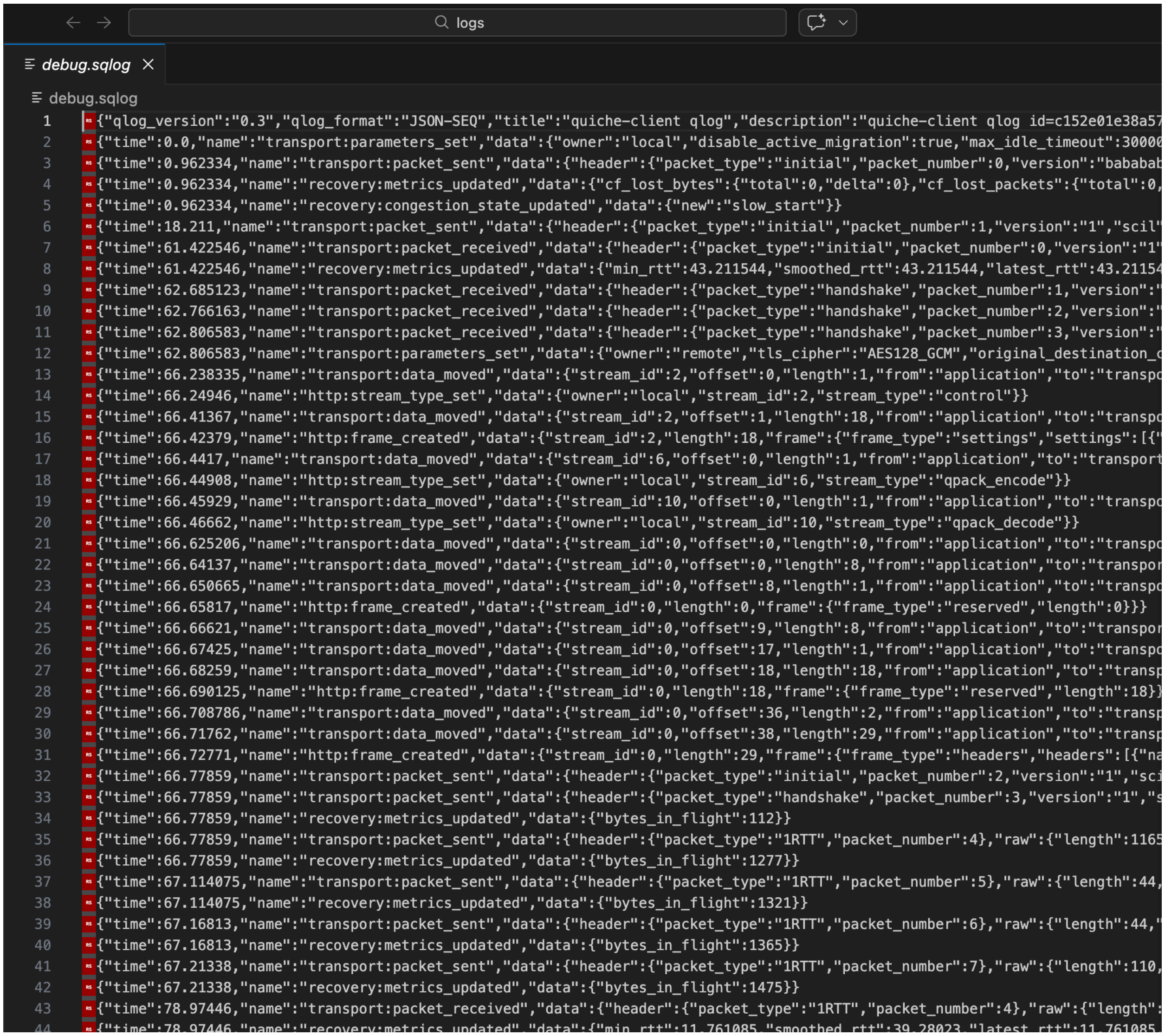
(using Chrome QUIC) (quic_google_01.pcap) (chrome_keys.log)



(using quiche) (quiche_traffic.pcap) (quiche_keys.log) (debug.sqlog)



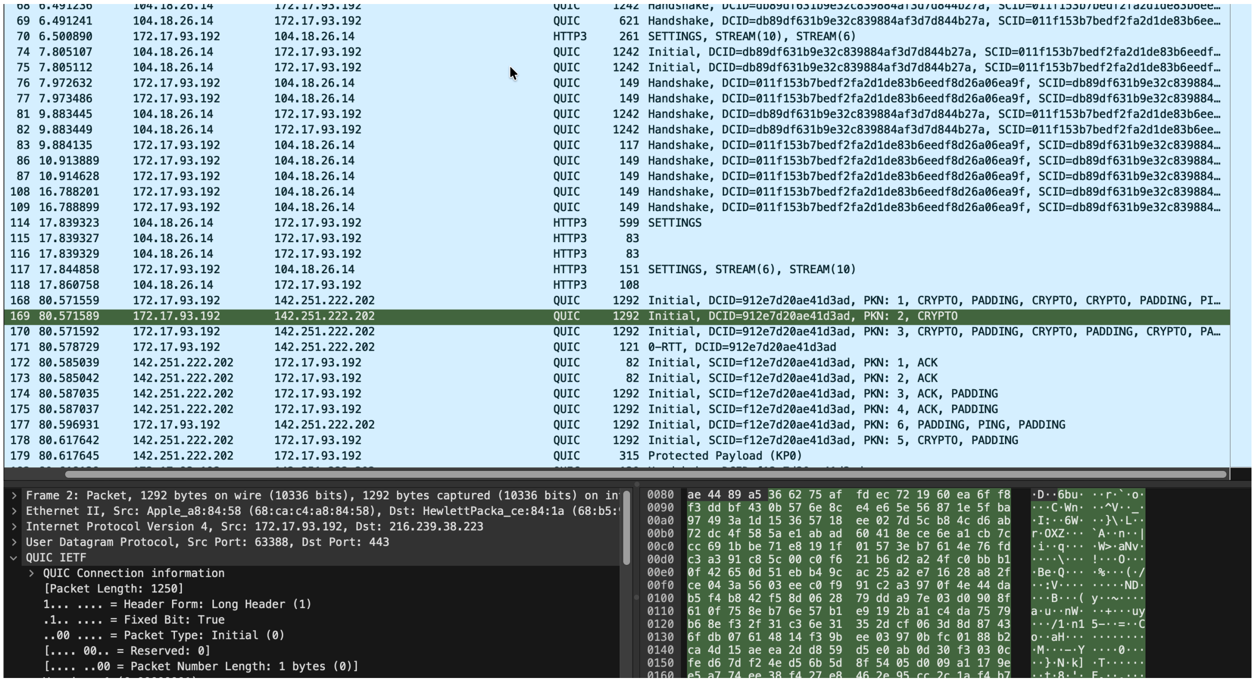
SQLOG File:



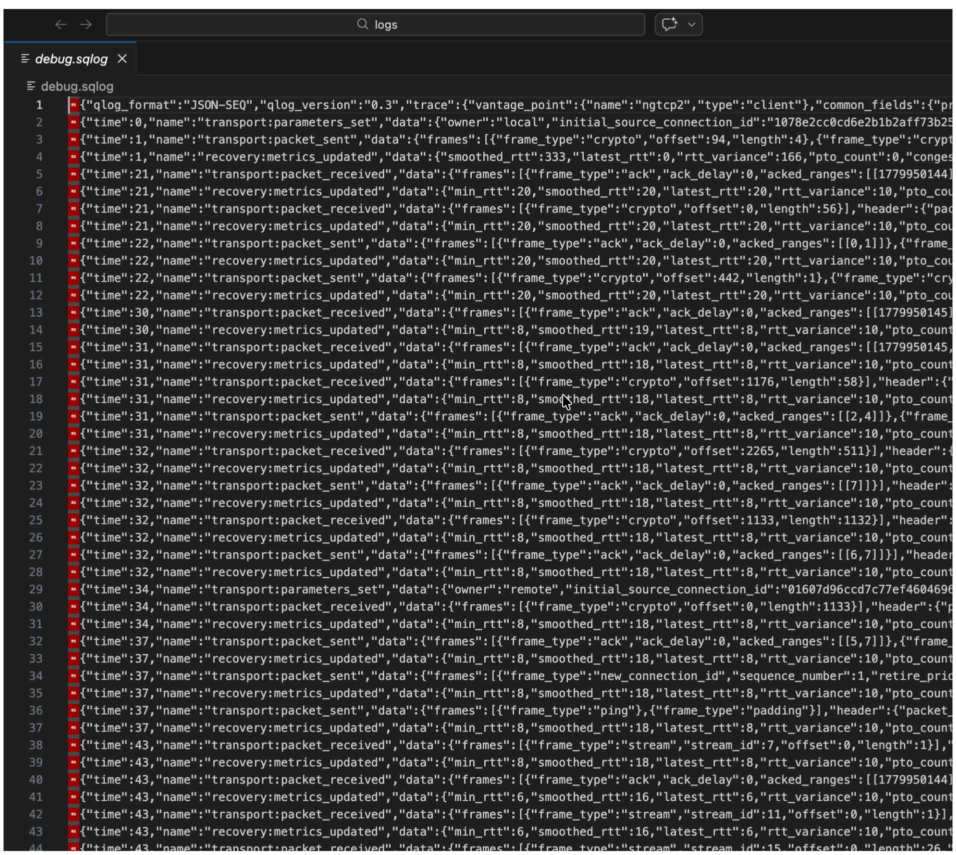
debug.sqlog


```
{"qlog_version":"0.3","qlog_format":"JSON-SEQ","title":"quiche-client qlog","description":"quiche-client qlog id=c152e01e38a57
{"time":0.0,"name":"transport:parameters_set","data":{"owner":"local","disable_active_migration":true,"max_idle_timeout":30000
{"time":0.962334,"name":"transport:packet_sent","data":{"header":{"packet_type":"initial","packet_number":0,"version":"bababab
{"time":0.962334,"name":"recovery:metrics_updated","data":{"cf_lost_bytes":{"total":0,"delta":0},"cf_lost_packets":{"total":0,
{"time":0.962334,"name":"recovery:congestion_state_updated","data":{"new":"slow_start"}}
{"time":18.211,"name":"transport:packet_sent","data":{"header":{"packet_type":"initial","packet_number":1,"version":"1","scil"
{"time":61.422546,"name":"transport:packet_received","data":{"header":{"packet_type":"initial","packet_number":0,"version":"1"
{"time":61.422546,"name":"recovery:metrics_updated","data":{"min_rtt":43.211544,"smoothed_rtt":43.211544,"latest_rtt":43.21154
{"time":62.685123,"name":"transport:packet_received","data":{"header":{"packet_type":"handshake","packet_number":1,"version":"
{"time":62.766163,"name":"transport:packet_received","data":{"header":{"packet_type":"handshake","packet_number":2,"version":"
{"time":62.806583,"name":"transport:packet_received","data":{"header":{"packet_type":"handshake","packet_number":3,"version":"
{"time":62.806583,"name":"transport:parameters_set","data":{"owner":"remote","tls_cipher":"AES128_GCM","original_destination_c
{"time":66.238335,"name":"transport:data_moved","data":{"stream_id":2,"offset":0,"length":1,"from":"application","to":"transpo
{"time":66.24946,"name":"http:stream_type_set","data":{"owner":"local","stream_id":2,"stream_type":"control"}}
{"time":66.41367,"name":"transport:data_moved","data":{"stream_id":2,"offset":1,"length":18,"from":"application","to":"transpo
{"time":66.42379,"name":"http:frame_created","data":{"stream_id":2,"length":18,"frame":{"frame_type":"settings","settings":[{"
{"time":66.4417,"name":"transport:data_moved","data":{"stream_id":6,"offset":0,"length":1,"from":"application","to":"transport
{"time":66.44908,"name":"http:stream_type_set","data":{"owner":"local","stream_id":6,"stream_type":"qpack_encode"}}
{"time":66.45929,"name":"transport:data_moved","data":{"stream_id":10,"offset":0,"length":1,"from":"application","to":"transpo
{"time":66.46662,"name":"http:stream_type_set","data":{"owner":"local","stream_id":10,"stream_type":"qpack_decode"}}
{"time":66.625206,"name":"transport:data_moved","data":{"stream_id":0,"offset":0,"length":0,"from":"application","to":"transpo
{"time":66.64137,"name":"transport:data_moved","data":{"stream_id":0,"offset":0,"length":8,"from":"application","to":"transpor
{"time":66.650665,"name":"transport:data_moved","data":{"stream_id":0,"offset":8,"length":1,"from":"application","to":"transpo
{"time":66.65817,"name":"http:frame_created","data":{"stream_id":0,"length":0,"frame":{"frame_type":"reserved","length":0}}}
{"time":66.66621,"name":"transport:data_moved","data":{"stream_id":0,"offset":9,"length":8,"from":"application","to":"transpor
{"time":66.67425,"name":"transport:data_moved","data":{"stream_id":0,"offset":17,"length":1,"from":"application","to":"transpo
{"time":66.68259,"name":"transport:data_moved","data":{"stream_id":0,"offset":18,"length":18,"from":"application","to":"transp
{"time":66.690125,"name":"http:frame_created","data":{"stream_id":0,"length":18,"frame":{"frame_type":"reserved","length":18}}
{"time":66.708786,"name":"transport:data_moved","data":{"stream_id":0,"offset":36,"length":2,"from":"application","to":"transp
{"time":66.71762,"name":"transport:data_moved","data":{"stream_id":0,"offset":38,"length":29,"from":"application","to":"transp
{"time":66.72771,"name":"http:frame_created","data":{"stream_id":0,"length":29,"frame":{"frame_type":"headers","headers":[{"na
{"time":66.77859,"name":"transport:packet_sent","data":{"header":{"packet_type":"initial","packet_number":2,"version":"1","sci
{"time":66.77859,"name":"transport:packet_sent","data":{"header":{"packet_type":"handshake","packet_number":3,"version":"1","s
{"time":66.77859,"name":"recovery:metrics_updated","data":{"bytes_in_flight":112}}
{"time":66.77859,"name":"transport:packet_sent","data":{"header":{"packet_type":"1RTT","packet_number":4},"raw":{"length":1165
{"time":66.77859,"name":"recovery:metrics_updated","data":{"bytes_in_flight":1277}}
{"time":67.114075,"name":"transport:packet_sent","data":{"header":{"packet_type":"1RTT","packet_number":5},"raw":{"length":44,
{"time":67.114075,"name":"recovery:metrics_updated","data":{"bytes_in_flight":1321}}
{"time":67.16813,"name":"transport:packet_sent","data":{"header":{"packet_type":"1RTT","packet_number":6},"raw":{"length":44,"
{"time":67.16813,"name":"recovery:metrics_updated","data":{"bytes_in_flight":1365}}
{"time":67.21338,"name":"transport:packet_sent","data":{"header":{"packet_type":"1RTT","packet_number":7},"raw":{"length":110,
{"time":67.21338,"name":"recovery:metrics_updated","data":{"bytes_in_flight":1475}}
{"time":78.97446,"name":"transport:packet_received","data":{"header":{"packet_type":"1RTT","packet_number":4},"raw":{"length":
{"time":78.97446,"name":"recovery:metrics_updated","data":{"min_rtt":11.761085,"smoothed_rtt":39.28023,"latest_rtt":11.761085,
```

(using ngtcp2) (ngtcp2_traffic.pcap) (ngtcp2_keys.log) (debug.sqlog)

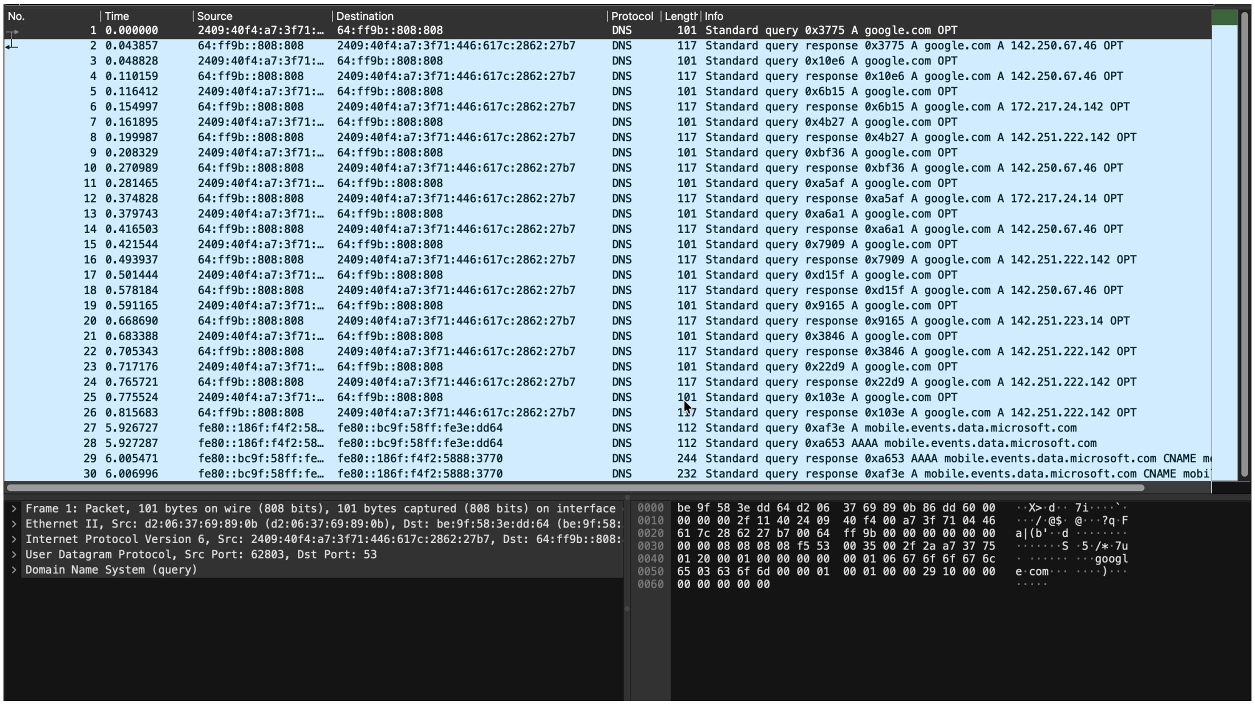


SQLOG File:



NON-QUIC DATA

DNS

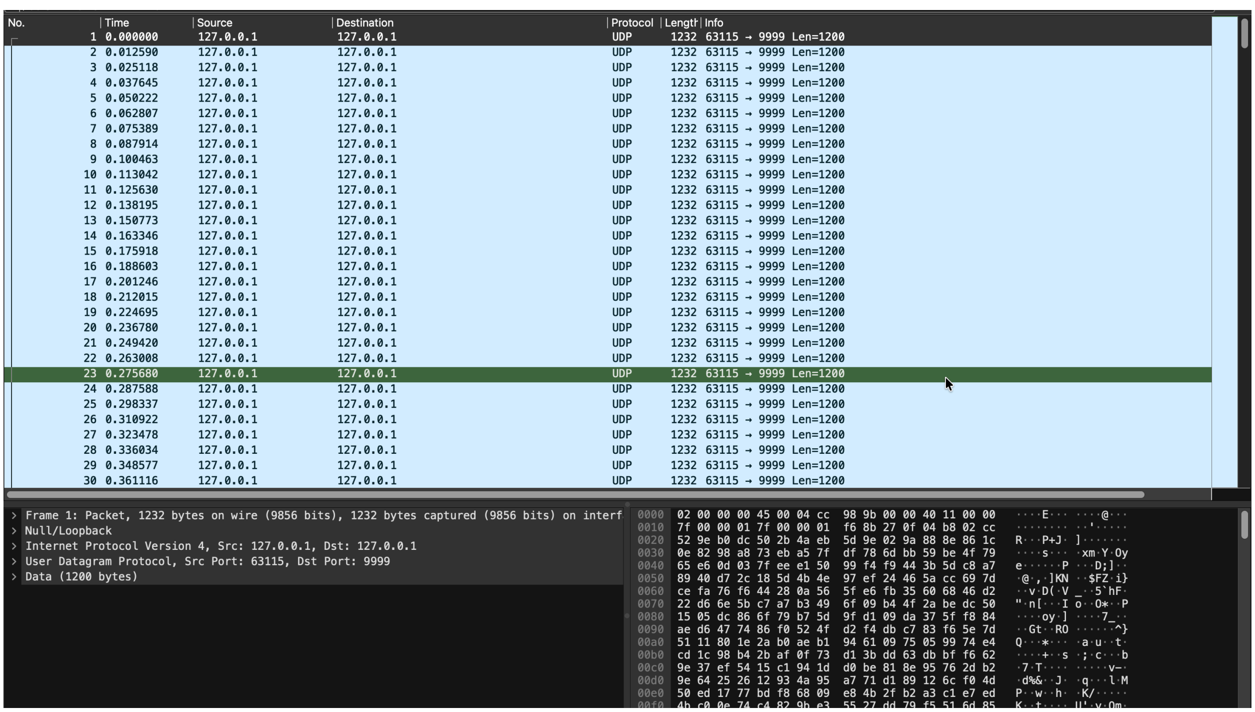


No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0x3775 A google.com OPT
2	0.043857	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0x3775 A google.com A 142.250.67.46 OPT
3	0.048828	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0x10e6 A google.com OPT
4	0.110159	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0x10e6 A google.com A 142.250.67.46 OPT
5	0.116412	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0x6b15 A google.com OPT
6	0.154997	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0x6b15 A google.com A 172.217.24.142 OPT
7	0.161895	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0x4b27 A google.com OPT
8	0.199987	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0x4b27 A google.com A 142.251.222.142 OPT
9	0.208329	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0xbf36 A google.com OPT
10	0.270989	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0xbf36 A google.com A 142.250.67.46 OPT
11	0.281465	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0xa5af A google.com OPT
12	0.374828	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0xa5af A google.com A 172.217.24.14 OPT
13	0.379743	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0xa6a1 A google.com OPT
14	0.416503	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0xa6a1 A google.com A 142.250.67.46 OPT
15	0.421544	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0x7909 A google.com OPT
16	0.493937	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0x7909 A google.com A 142.251.222.142 OPT
17	0.501444	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0xd15f A google.com OPT
18	0.578184	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0xd15f A google.com A 142.250.67.46 OPT
19	0.591165	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0x9165 A google.com OPT
20	0.668690	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0x9165 A google.com A 142.251.223.14 OPT
21	0.683388	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0x3846 A google.com OPT
22	0.705343	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0x3846 A google.com A 142.251.222.142 OPT
23	0.717176	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0x22d9 A google.com OPT
24	0.765721	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0x22d9 A google.com A 142.251.222.142 OPT
25	0.775524	2409:40f4:a7:3f71:…	64:ff9b::808:808	DNS	101	Standard query 0x103e A google.com OPT
26	0.815683	64:ff9b::808:808	2409:40f4:a7:3f71:446:617c:2862:27b7	DNS	117	Standard query response 0x103e A google.com A 142.251.222.142 OPT
27	5.926727	fe80::186f:f4f2:58…	fe80::bc9f:58ff:fe3e:dd64	DNS	112	Standard query 0xaf3e A mobile.events.data.microsoft.com
28	5.927287	fe80::186f:f4f2:58…	fe80::bc9f:58ff:fe3e:dd64	DNS	112	Standard query 0xa653 AAAA mobile.events.data.microsoft.com
29	6.005471	fe80::bc9f:58ff:fe…	fe80::186f:f4f2:5888:3770	DNS	244	Standard query response 0xa653 AAAA mobile.events.data.microsoft.com CNAME m
30	6.006996	fe80::bc9f:58ff:fe…	fe80::186f:f4f2:5888:3770	DNS	232	Standard query response 0xaf3e A mobile.events.data.microsoft.com CNAME mobi

```
> Frame 1: Packet, 101 bytes on wire (808 bits), 101 bytes captured (808 bits) on interface
> Ethernet II, Src: d2:06:37:69:89:0b (d2:06:37:69:89:0b), Dst: be:9f:58:3e:dd:64 (be:9f:58:
> Internet Protocol Version 6, Src: 2409:40f4:a7:3f71:446:617c:2862:27b7, Dst: 64:ff9b::808:
> User Datagram Protocol, Src Port: 62803, Dst Port: 53
> Domain Name System (query)
```

```
0000  be 9f 58 3e dd 64 d2 06  37 69 89 0b 86 dd 60 00
0010  00 00 00 2f 11 40 24 09  40 f4 00 a7 3f 71 04 46
0020  61 7c 28 62 27 b7 00 64  ff 9b 00 00 00 00 00 00
0030  00 00 08 08 08 08 f5 53  00 35 00 2f 2a a7 37 75
0040  01 20 00 01 00 00 00 00  00 01 06 67 6f 6f 67 6c
0050  65 03 63 6f 6d 00 00 01  00 01 00 00 29 10 00 00
0060  00 00 00 00 00
```

Randome Noise



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
2	0.012590	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
3	0.025118	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
4	0.037645	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
5	0.050222	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
6	0.062807	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
7	0.075389	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
8	0.087914	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
9	0.100463	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
10	0.113042	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
11	0.125630	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
12	0.138195	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
13	0.150773	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
14	0.163346	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
15	0.175918	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
16	0.188603	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
17	0.201246	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
18	0.212015	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
19	0.224695	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
20	0.236780	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
21	0.249420	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
22	0.263008	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
23	0.275680	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
24	0.287588	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
25	0.298337	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
26	0.310922	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
27	0.323478	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
28	0.336034	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
29	0.348577	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200
30	0.361116	127.0.0.1	127.0.0.1	UDP	1232	63115 → 9999 Len=1200

```
> Frame 1: Packet, 1232 bytes on wire (9856 bits), 1232 bytes captured (9856 bits) on interf
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> User Datagram Protocol, Src Port: 63115, Dst Port: 9999
> Data (1200 bytes)
```

```
0000  02 00 00 00 45 00 04 cc  98 9b 00 00 40 11 00 00
0010  7f 00 00 01 7f 00 00 01  f6 8b 27 0f 04 b8 02 cc
0020  52 9e b0 dc 50 2b 4a eb  5d 9e 02 9a 88 8e 86 1c
0030  0e 82 98 a8 73 eb a5 7f  df 78 6d bb 59 be 4f 79
0040  65 e6 0d 03 7f ee e1 50  99 f4 f9 44 3b 5d c8 a7
0050  89 40 d7 2c 18 5d 4b 4e  97 ef 24 46 5a cc 69 7d
0060  ce fa 76 f6 44 28 0a 56  5f e6 fb 35 60 68 46 d2
0070  22 d6 6e 5b c7 a7 b3 49  6f 09 b4 4f 2a be dc 50
0080  15 05 dc 86 6f 79 b7 5d  9f d1 09 da 37 5f f8 84
0090  ae d6 47 74 86 f0 52 4f  d2 f4 db c7 83 f6 5e 7d
00a0  51 11 80 1e 2a b0 ae b1  94 61 09 75 05 99 74 e4
00b0  cd 1c 98 b4 2b af 0f 73  d1 3b dd 63 db bf f6 62
00c0  9e 37 ef 54 15 c1 94 1d  d0 be 81 8e 95 76 2d b2
00d0  9e 64 25 26 12 93 4a 95  a7 71 d1 89 12 6c f0 4d
00e0  50 ed 17 77 bd f8 68 09  e8 4b 2f b2 a3 c1 e7 ed
```

QoS MECHANISMS

1. Classification and Marking: Identify and label packets based on their type.

Incoming Packet
↓
[Classifier] - "Is this video? web? file transfer?"
↓
[Marker] - Add a priority label
↓
Marked Packet (now has priority info)

Common marking methods:

DSCP (Differentiated Services Code Point):

- 6-bit field in IP header
- Values: 0-63
- Examples:
  - DSCP 46 (EF) = Expedited Forwarding (highest priority - voice)
  - DSCP 34 (AF41) = Assured Forwarding (high priority - video)
  - DSCP 0 (BE) = Best Effort (lowest priority - bulk data)

802.1p (Layer 2 marking):

- 3-bit field in Ethernet header
- Values: 0-7 (7 = highest priority)

2. Queueing Disciplines

- **FIFO (First-In-First-Out):**
  - Simplest: no prioritization
  - Not suitable for differentiated service
- **Priority Queuing:**
  - Multiple queues with strict priority
  - Risk: lower priority starvation
- **Weighted Fair Queuing (WFQ):**
  - Bandwidth sharing based on weights
  - Guarantees minimum bandwidth per class - Recommended for QUIC traffic

3. Congestion Avoidance

- **RED (Random Early Detection):**
    - Drops packets before queue is full
    - Prevents TCP global synchronization
    - Less relevant for QUIC (has own congestion control)
- **WRED (Weighted RED):**
    - Different drop thresholds per traffic class
    - Higher priority traffic dropped last

POLICING MECHANISMS

Feature	Policing	Shaping
Action	Drops packets immediately if they exceed the rate.	Buffers packets in a queue to send them later.
Effect	Causes instant packet loss; "choppy" traffic.	Introduces Latency (Delay); "smooth" traffic.
Visual	The traffic graph looks "clipped" at the top.	The traffic graph looks like a "smooth wave."
Usage	Common at ISP entry points (Ingress).	Common at your router's exit (Egress).

1) Token Bucket Algorithm:

Mechanism:
- Tokens generated at constant rate R
- Bucket capacity B (burst size)
- Packet needs tokens equal to its size
- No tokens = packet dropped

Advantages:
- Allows controlled bursts
- Simple to implement
- Low computational overhead

2) Leaky Bucket Algorithm

Mechanism:

- Packets enter at variable rate
- Leak out at constant rate R
- Bucket capacity B

Characteristics:

- Smooths all traffic bursts
- Output always constant rate
- More restrictive than token bucket

When to use:

- When constant output rate required
- Protecting downstream with strict rate limits
- Not ideal for QUIC (prevents legitimate bursts)

3) Dual Token Bucket (Two-Rate Policer)

Mechanism:

- CIR bucket (Committed Information Rate)
- PIR bucket (Peak Information Rate)
- Three-color marking: GREEN, YELLOW, RED

Marking Logic:

Packet size ≤ CIR tokens → GREEN (forward, high priority) CIR tokens < Packet ≤ PIR → YELLOW (forward, medium priority) Packet > PIR tokens → RED (drop or lowest priority)

Use Case:

- Enforce committed vs. burst rates
- Graceful degradation (YELLOW marked packets)
- More flexible than single token bucket

4) DiffServ Architecture

Most widely deployed QoS framework:

- Edge classification: Traffic identified and marked
- Core forwarding: Routers treat packets based on DSCP
- Scalable: No per-flow state in core